

Archivio XML dei CIG ANAC

Relazione sintetica sulla preparazione documentale

Massimiliano Ricchiuti
Elaborazione di testi e gestione documentale

A.A. 2025/2026

1. Obiettivo e dataset

Il progetto realizza una pipeline Python che processa fonti relative a contratti pubblici, le trasforma in XML, valida i documenti mediante DTD, estrae informazioni e genera una collezione HTML navigabile. Il dataset comprende 21 XML, oltre la soglia minima di 15 documenti del progetto individuale, derivati da fonti CSV, JSON, HTML e PDF conservate in `fonti_originali/`. A queste si affiancano 33 risorse web istituzionali o primarie, qualificate e riferite a tutti i CIG. Il campione è didattico e non rappresenta l'intero universo ANAC.

La preparazione parte dai CIG presenti nei JSON, integra le righe omologhe del CSV e collega le ulteriori fonti locali. L'associazione non usa eccezioni dedicate: cerca un CIG noto nel nome e, se il nome è descrittivo, nel testo estraibile. Così il disciplinare di Frascati viene associato a B6DD95EE23 perché il codice compare nel testo. Le risorse online sono invece governate da un manifesto: per ciascuna conserva URL, ente, formato, fase, sintesi e nesso probatorio. Il caricamento fallisce se manca un CIG o un metadato.

2. Modello XML e validazione

Ogni radice `contratto` reca il CIG e contiene fonti locali, approfondimenti web, informazioni di gara, stazione appaltante, pubblicazioni, partecipanti, importi, date, CPV e descrizione documentale. I file locali conservano tipo, MIME type e percorso; le risorse online mantengono inoltre URL, relazione e data di verifica.

La DTD `schema/contratto_cig.dtd` stabilisce ordine, cardinalità e attributi. Il requisito del contenuto testuale è soddisfatto da un content model misto conforme alla sintassi XML:

```
<!ELEMENT descrizioneDocumentale
  (#PCDATA | tipologia | oggetto | categoria | termine | luogo)*>
```

Un secondo modello applica lo stesso principio alle fonti esterne:

```
<!ELEMENT fonteWeb
  (#PCDATA | titoloFonte | enteFonte | faseFonte | nessoFonte | datoFonte)*>
```

I due elementi alternano testo ed elementi semantici ripetibili; il corpus non è quindi composto soltanto da blocchi strutturati. La build valida tutti gli XML con `lxml`; un documento mal formato o non valido interrompe l'esecuzione ed è registrato in `output_data/validation.xml`.

3. Estrazione, analisi e output

Gli XPath convertono gli XML validi in istanze del modello intermedio `ContractRecord`. Su tale modello sono calcolate distribuzioni territoriali, copertura dei campi, sequenze cronologiche, statistiche economiche e frequenze testuali. I PDF sono esaminati con `pypdf`, con ripiego su `pdfminer.six`; gli importi usano `Decimal` e le incongruenze temporali restano visibili senza correzioni arbitrarie.

Jinja2 genera `index.html`, un archivio filtrabile, una pagina per ciascun XML valido, l'analisi, la qualità dei dati e la sezione *Progetto e metodo* integrata nella home. Le schede mostrano tutti i campi XML e consentono di esplorare l'albero e leggere o scaricare il sorgente. Ogni dettaglio separa i download locali dalle fonti web verificate e mostra per queste ultime ente, fase, sintesi e ragione del collegamento. Per B6DD95EE23 il fascicolo ufficiale espone determina iniziale, disciplinare, verbale, graduatoria ed esito. Le pagine recano microdata `schema.org/GovernmentService`; CSS, JavaScript e grafici SVG sono locali e responsivi.

4. Riproducibilità, verifiche e uso di strumenti AI

I comandi principali sono:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt -r requirements-dev.txt
python scripts/build_all.py
python -m pytest
```

La suite verifica parsing, DTD, uso dei due modelli misti, copertura 21/21 del manifesto, estrazione, statistiche, mapping dei documenti descrittivi, pagine, microdata e collegamenti interni. GitHub Actions ripete build e test a ogni push su `main` o `development` e pubblica `dist/` su GitHub Pages.

Gli strumenti AI sono stati impiegati per la revisione architetturale, l'implementazione e il controllo della documentazione. Le proposte sono state sottoposte a validazione DTD, test automatici, build completa e ispezione dell'output. I prompt e le finalità d'uso sono riportati in `report/prompts_utilizzati.md`. Le fonti originali, gli script di preparazione, la DTD, gli XML e gli output HTML restano disponibili e ispezionabili.

5. Riscontro dei requisiti

- 21 file XML distinti e validi, con directory di input configurabile;
- DTD creata per il progetto e due content model misti effettivamente usati;
- fonti eterogenee CSV, JSON, HTML, PDF e 33 risorse web qualificate, con copertura 21/21;
- una pagina HTML per ogni XML, indice navigabile e pagine aggiuntive di analisi;
- download dell'XML e dei documenti originali collegati al relativo CIG;
- foglio CSS comune, layout responsivo e annotazioni microdata generate;
- relazione entro tre pagine e prompt conservati nella documentazione.

6. Limiti

Il dataset è circoscritto e presenta campi mancanti o semanticamente non omogenei; perciò le analisi sono descrittive. Il riconoscimento locale richiede il CIG nel nome o nel testo estraibile: i PDF immagine necessitano di OCR. Le pagine web possono inoltre mutare dopo la data di verifica. Per questo ogni nesso è classificato: lotto, accordo quadro, CUP, fase antecedente o contesto non sono presentati come atti con CIG esatto. I limiti restano dichiarati e non sono colmati con dati inventati.